Operating Systems Note

Contents

1 Introduction **1**

2 Interrupts (IVEC - Interrupt Vector) **1**

3 x86 rings **2**

4 Booting Process **2**

4.1 Master Boot Record (MBR) 2

1 Introduction

- **Consider a machine with single core CPU with one program counter and one instruction register.**
- When an application is running it runs directly on the hardware, not on top of the OS. (It is called running on bare metal).
- So according to that, if anything is currently running, then it should be on bare metal.

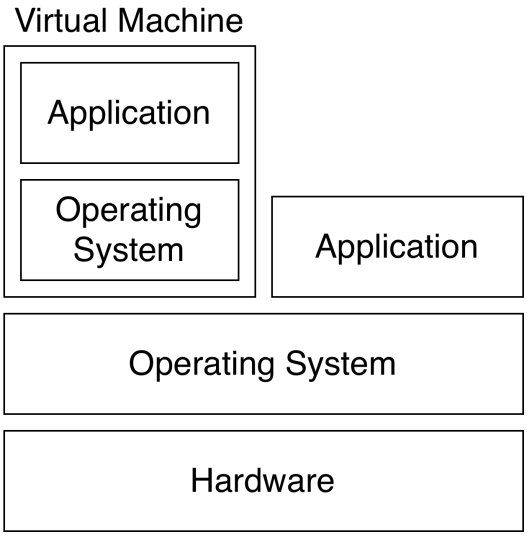


Figure 1: This is the normal depiction

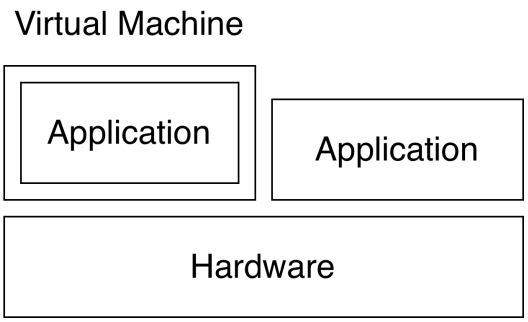
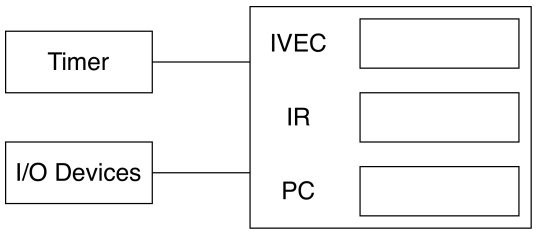


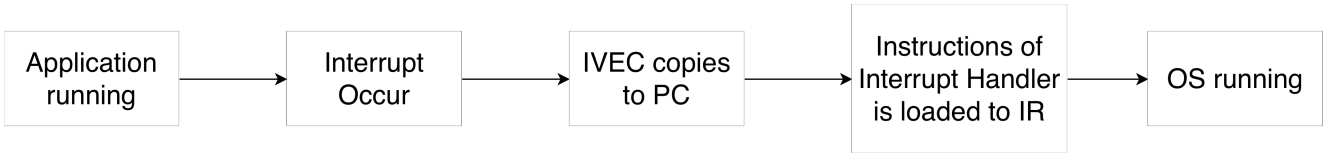
Figure 2: This is whats actually happening

- Main memory is accessed in bytes (byte addressable), but the hard disk is accessed as sectors since hard disk is slower than main memory an waiting for one byte is just a waste of time for the CPU. Therefore memory on hard disk is accessed by slots of bytes called sectors.
- These memory is typically depicted as an array for ease of understanding.

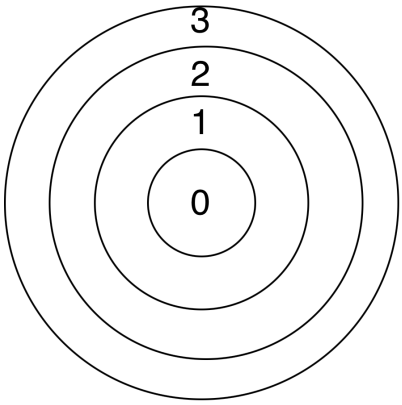
2 Interrupts (IVEC - Interrupt Vector)



- Interrupts are signals generated by IO devices like keyboard and mouse or the timer. These devices are connected to the CPU using interrupt lines.
- IVEC register is a type of register which contains the memory location of the starting address of the interrupt handlers first instruction which is stored with the OS.
- When an interrupt signal is received the address is IVEC is copied into the PC, which makes the instruction in IR to jump to that location in memory which contains the OS. This helps to prevent an application from hogging the CPU.
- If any keyboard or mouse interruption does not occur, the application may run forever hogging all the hardware. To prevent this the timer makes an interruption in regular time periods.



3 x86 rings



- Depicts the privilege levels of different processes. 0 has the highest privilege while 3 has the lowest.
- This helps to prevent buggy software from crashing the system by preventing memory access, execution permission and instruction execution with different privileges.
- **Ring 0**: Kernel Mode. This is where the OS runs with full unrestricted access to memory and other hardware devices.
- **Ring 1 & 2**: Designed for device drivers and OS services which requires intermediate privileges.
- **Ring 3**: User Mode. Applications, scripts and user-space processes execute here with restricted access to hardware and isolated virtual memory.

4 Booting Process

- Just when the computer is turned on, memory and all the registers contain some random numbers, since they are all binary devices (must contains 0 or 1).
- So at the start, CPU is executing some random instructions. To prevent this PC is hardcoded to some memory location which is on the ROM (ROM is non-volatile memory. Unlike RAM it's data aren't lost after losing electricity).
- The instructions they run now are the BIOS stored in the ROM. All these instructions are executed with the highest privilege.
- In real x86 architecture, the PC is hardcoded to a memory address which is 16 bytes below the highest memory address. This 16 bytes contains a jump instruction to jump the PC to the start of the ROM. This can help to deal with different ROM sizes in different computers.
- At the start computer runs the POST (Power On Self Test) program stored in the BIOS, which check whether all the hardware devices are attached and is working properly.
- Next it reads the first sector of the hard disk to the RAM which usually contains a boot sector. After loading the data PC is jumped to the starting address of that copied data in the RAM (the Boot Call), after that the CPU keeps executing and loading the next boot sector until the booting process is done.

4.1 Master Boot Record (MBR)

- Consider a hard disk with sectors containing 512 bytes of data.
- The first 446 bytes will contain the Boot Code, the next 64 bytes will contain the partition table and the last 2 bytes will contain the magic number which indicates that the given sector is a boot sector (0x55AA).

  EXTRA: In linux, the kernel uses magic bytes to decide how to run executables and the file command identifies the files by magic number. The extension doesn't matter in here.

  Ex: for Java class files → CAFEBABE

  Ex: for bash scripts → #!/bin/bash